

MANUAL DE BASE DE DATOS

Aplicativo de Gestión de Préstamos y Reservas de Libros



AUTORES:

- Wendy Yolanny Jaimes Lugo
- Beatriz Adriana Pérez Orozco
- Jonathan David Quintero Villamizar

Contenido

1. Introducción	3
2. Elección del SGBD	4
3. Glosario	5
4. Modelo Entidad-Relación	6
Reglas de Integridad Estructural y de Negocio:	6
5. Tablas y Estructuras de Datos.....	7
Tabla: public.profiles	7
Tabla: public.books	8
Tabla: public.loans	9
Tabla: public.reservations	10
6. Políticas de Seguridad (RLS) y Control de Acceso	12
Políticas aplicadas a Profiles (Perfiles).....	12
Políticas aplicadas a User Roles (Roles de Usuario).....	12
Políticas aplicadas a Books (Catálogo de Libros)	12
Políticas aplicadas a Loans y Reservations (Préstamos y Reservas)	13
7. Triggers y Automatizaciones	14
A. Automatización de Auditoría Temporal	14
B. Orquestación Automática en el Registro de Usuarios (Onboarding).....	14

1. Introducción

El presente documento tiene como propósito ofrecer una visión integral y formal de la arquitectura de datos del proyecto, el cual emplea una base de datos relacional avanzada implementada bajo el motor **PostgreSQL**, integrada de manera nativa mediante la plataforma de servicios en la nube **Supabase**. Esta infraestructura constituye el núcleo operativo fundamental para la persistencia, consistencia y auditoría del sistema enfocado en la administración de un catálogo bibliográfico, la trazabilidad de préstamos físicos y la gestión automatizada de reservas de libros para usuarios autenticados.

En este manual se detalla minuciosamente el modelo entidad-relación que subyace a la lógica de negocio. Este modelo ofrece una representación conceptual sólida de las entidades clave, sus atributos descriptivos, restricciones de integridad referencial y mecanismos avanzados de aislamiento de datos. Proporciona una comprensión clara y organizada de cómo se estructuran e interconectan los datos, permitiendo un análisis fluido de las dependencias relacionales y asegurando la escalabilidad del ecosistema informático.

El alcance de esta documentación abarca desde la tipificación técnica de cada tabla hasta la configuración explícita de directivas de seguridad a nivel de fila (Row Level Security - RLS). Asimismo, se desglosan las reglas de negocio automatizadas a través de disparadores (Triggers) y funciones seguras (Security Definer), facilitando con ello un marco estandarizado para el desarrollo continuo, el mantenimiento preventivo y la correcta administración de la base de datos por parte de ingenieros, administradores y partes interesadas.

2. Elección del SGBD

La selección de **PostgreSQL** como Sistema de Gestión de Bases de Datos Relacionales (SGBD) para este proyecto obedece a criterios estrictos de rendimiento, seguridad y adaptabilidad frente a arquitecturas web modernas, superando las prestaciones de otras alternativas tradicionales:

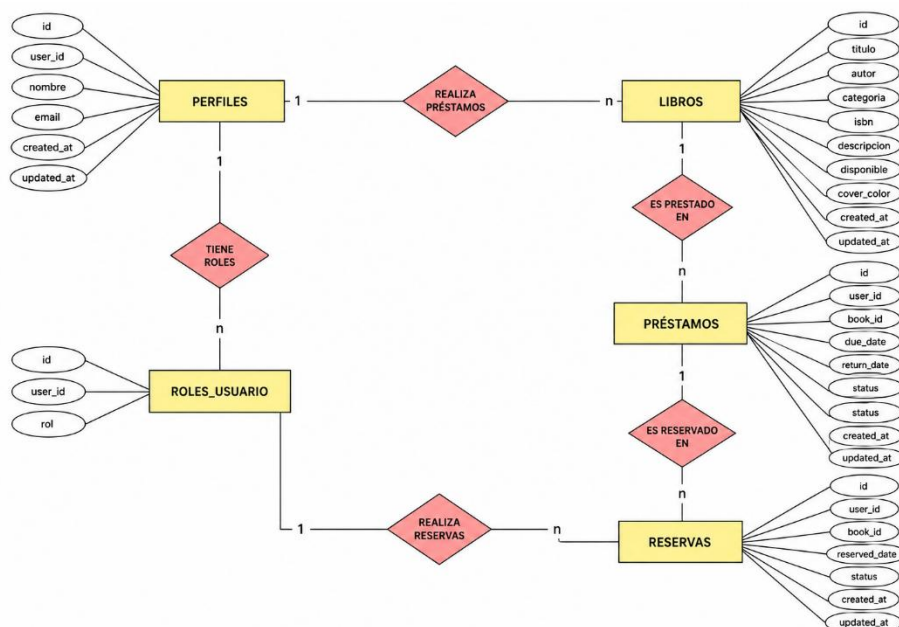
- **Soporte Nativo de Tipos Complejos:** Permite el uso eficiente de Identificadores Universales Únicos (UUID) generados algorítmicamente mediante `gen_random_uuid()`, evitando las colisiones y limitaciones de las secuencias auto-incrementales tradicionales. Asimismo, incorpora tipos cronológicos con zona horaria integrada (`TIMESTAMPTZ`) y tipos enumerados personalizados (`CREATE TYPE AS ENUM`).
- **Seguridad Integrada en el Motor (RLS):** A diferencia de otros gestores que delegan el aislamiento de registros exclusivamente a la capa de aplicación (Backend), PostgreSQL permite declarar directivas de *Row Level Security* de manera nativa, garantizando que ninguna consulta externa pueda saltarse las reglas de acceso declaradas.
- **Robustez Transaccional y ACID:** Cumple estrictamente con los principios de Atomicidad, Consistencia, Aislamiento y Durabilidad, garantizando operaciones concurrentes seguras y libres de condiciones de carrera (por ejemplo, al decrementar existencias o actualizar estados concurrentes de una reserva).
- **Integración Fluida con Proveedores Modernos:** Su compatibilidad directa con Supabase facilita el aprovisionamiento automático de esquemas de autenticación, la gestión de ganchos web (Webhooks) y la sincronización asíncrona de eventos de sesión con las tablas de negocio.

3. Glosario

- **PostgreSQL:** Sistema de gestión de bases de datos relacional de código abierto y orientado a objetos, ampliamente reconocido por su extensibilidad, fiabilidad y estricto cumplimiento de estándares SQL.
- **UUID (Universally Unique Identifier):** Estructura de datos de 128 bits empleada para identificar de forma unívoca y global un registro dentro de un sistema distribuido sin requerir coordinación centralizada.
- **RLS (Row Level Security):** Característica de seguridad que aplica políticas de filtrado de filas directamente sobre las consultas SQL que se ejecutan, basándose en la identidad o el rol del usuario que realiza la petición.
- **Enum (Tipo Enumerado):** Objeto estático en la base de datos que define una lista explícita y predeterminada de valores permitidos para una columna, previniendo la inserción de datos inconsistentes.
- **Trigger (Disparador):** Mecanismo que evalúa y ejecuta un bloque de funciones procedurales de manera automática inmediatamente antes o después de que acontezca un evento de manipulación de datos (INSERT, UPDATE, DELETE).
- **Security Definer:** Propiedad de una función SQL que le permite ejecutarse utilizando los privilegios y permisos del usuario que creó la función, en lugar de los privilegios del usuario que la invoca.
- **TIMESTAMPTZ:** Tipo de dato temporal de PostgreSQL que almacena la fecha y la hora exacta normalizada en UTC, convirtiéndola automáticamente a la zona horaria del cliente en cada consulta.

4. Modelo Entidad-Relación

La base de datos organiza sus estructuras de negocio bajo el esquema por defecto `public`, interactuando de forma controlada y relacional con el esquema interno de identidad de la plataforma (`auth`).



Reglas de Integridad Estructural y de Negocio:

- Cada cuenta registrada en el módulo de autenticación (`auth.users`) desencadena la creación en paralelo de un registro descriptivo en `public.profiles` y una asignación inicial de rol en `public.user_roles`.
- Las entidades transaccionales `public.loans` (préstamos) y `public.reservations` (reservas) actúan como puentes relacionales obligatorios entre la cuenta del usuario y un ejemplar disponible dentro del catálogo de libros (`public.books`).
- Se implementan restricciones de integridad referencial con propagación en cascada (`ON DELETE CASCADE`). Si un registro de usuario es eliminado del sistema, se depuran de forma automática e inmediata sus perfiles, roles, registros de préstamos y reservas asociadas para evitar la existencia de registros huérfanos.

5. Tablas y Estructuras de Datos

A continuación, se presenta la especificación técnica pormenorizada de cada una de las tablas que componen el esquema relacional de la aplicación:

Tabla: public.profiles

Contiene la información complementaria de uso público y administrativo de los usuarios inscritos en la plataforma.

Campo	Tipo de Dato	Descripción y Restricciones
id	UUID	Llave Primaria. Identificador unívoco generado por defecto mediante <code>gen_random_uuid()</code> .
user_id	UUID	Llave Foránea referente a <code>auth.users(id)</code> . Restricción UNIQUE y NOT NULL. Eliminación en cascada.
name	TEXT	Nombre completo del usuario. NOT NULL. Cadena vacía por defecto ('').
email	TEXT	Dirección de correo electrónico de contacto. NOT NULL. Cadena vacía por defecto.
created_at	TIMESTAMPTZ	Fecha y hora de inserción del registro. Capturada automáticamente vía <code>now()</code> .
updated_at	TIMESTAMPTZ	Fecha y hora de la última modificación estructural del registro.

Tabla: public.user_roles

Matriz intermedia encargada de asociar roles de acceso específicos a los usuarios para el control de permisos del aplicativo.

Campo	Tipo de Dato	Descripción y Restricciones
id	UUID	Llave Primaria. Identificador del registro autogenerado por defecto.
user_id	UUID	Llave Foránea vinculada a <code>auth.users(id)</code> . NOT NULL. Eliminación en cascada.
role	app_role (ENUM)	Rol asignado del tipo estructurado ('admin', 'user'). NOT NULL. Valor por defecto: 'user'.

Restricción de unicidad: Cuenta con un índice único compuesto: `UNIQUE (user_id, role)`, impidiendo duplicidad en la asignación del mismo rol a un usuario.

Tabla: public.books

Almacena la ficha técnica, metadatos esenciales y estado operativo de las obras literarias y académicas indexadas.

Campo	Tipo de Dato	Descripción y Restricciones
id	UUID	Llave Primaria. Identificador unívoco del libro.
title	TEXT	Título principal de la obra. NOT NULL.
author	TEXT	Nombre del autor o entidad creadora del volumen. NOT NULL.
category	TEXT	Categoría, género o clasificación bibliográfica. Por defecto cadena vacía.
isbn	TEXT	International Standard Book Number (Código de barras/registro).
description	TEXT	Sinopsis, resumen analítico o notas descriptivas del libro.
available	BOOLEAN	Indicador booleano de disponibilidad. NOT NULL. Por defecto: <code>true</code> .
cover_color	TEXT	Estilos o gradientes CSS almacenados para renderizado visual (Defecto: <code>'from-primary to-secondary'</code>).
created_at	TIMESTAMPZ	Fecha de incorporación del libro al catálogo.
updated_at	TIMESTAMPZ	Última actualización de metadatos o estado de disponibilidad.

Tabla: **public.loans**

Entidad transaccional dedicada a regular el estado de los préstamos temporales de libros realizados a los usuarios.

Campo	Tipo de Dato	Descripción y Restricciones
id	UUID	Llave Primaria. Identificador de control del préstamo.
user_id	UUID	Llave Foránea vinculada a <code>auth.users(id)</code> . NOT NULL.
book_id	UUID	Llave Foránea vinculada a <code>public.books(id)</code> . NOT NULL.
loan_date	DATE	Fecha de salida física del libro. NOT NULL. Por defecto: <code>CURRENT_DATE</code> .
due_date	DATE	Fecha límite programada para efectuar la entrega o devolución. NOT NULL.
return_date	DATE	Fecha real en la que el usuario regresó el libro. Permite valores nulos (NULL).
status	TEXT	Estado actual del flujo de préstamo. NOT NULL. Por defecto: <code>'active'</code> .
created_at	TIMESTAMPTZ	Sello cronológico de registro de la transacción en base de datos.
updated_at	TIMESTAMPTZ	Fecha de última actualización del estado o extensión del préstamo.

Validación CHECK del Estado: Columna `status` restringida mediante la regla: `CHECK (status IN ('active', 'returned', 'overdue'))`.

Tabla: `public.reservations`

Registra las solicitudes de apartamiento prioritario sobre libros que se encuentran actualmente en circulación o no disponibles.

Campo	Tipo de Dato	Descripción y Restricciones
id	UUID	Llave Primaria. Identificador único de la reserva.
user_id	UUID	Llave Foránea vinculada a <code>auth.users(id)</code> . NOT NULL.
book_id	UUID	Llave Foránea vinculada a <code>public.books(id)</code> . NOT NULL.
reserved_date	DATE	Fecha de radicación de la reserva. NOT NULL. Por defecto: <code>CURRENT_DATE</code> .
status	TEXT	Estado operacional de la reserva. NOT NULL. Por defecto: <code>'pending'</code> .
created_at	TIMESTAMPTZ	Fecha y hora de creación de la solicitud.
updated_at	TIMESTAMPTZ	Sello cronológico del último cambio de estado del registro.

Validación CHECK del Estado: Columna `status` restringida mediante la regla: `CHECK (status IN ('pending', 'ready', 'cancelled'))`.

6. Políticas de Seguridad (RLS) y Control de Acceso

El motor de la base de datos tiene activada la directiva `ENABLE ROW LEVEL SECURITY` en cada una de sus tablas principales. El control de acceso selectivo se evalúa de manera centralizada mediante la función optimizada de validación de roles de seguridad:

```
CREATE OR REPLACE FUNCTION public.has_role(_user_id UUID, _role app_role)
RETURNS BOOLEAN AS $$
    SELECT EXISTS (
        SELECT 1 FROM public.user_roles
        WHERE user_id = _user_id AND role = _role
    )
$$ LANGUAGE sql STABLE SECURITY DEFINER;
```

A continuación, se detallan las políticas que gobiernan la visibilidad y manipulación de los registros por cada entidad:

Políticas aplicadas a Profiles (Perfiles)

- **Lectura (SELECT):** Permitida a cualquier usuario autenticado en la plataforma de manera global (*"Anyone authenticated can view profiles"*).
- **Insertión y Actualización (INSERT, UPDATE):** Restringido explícitamente al usuario propietario de la cuenta mediante la comprobación estricta de identidad: `auth.uid() = user_id`.

Políticas aplicadas a User Roles (Roles de Usuario)

- **Lectura (SELECT):** Un usuario estándar solo está facultado para ver sus propios roles asignados (`auth.uid() = user_id`). El rol de administración tiene privilegios extendidos para consultar el mapa completo de roles invocando la función `public.has_role(auth.uid(), 'admin')`.
- **Control Pleno (ALL):** Las facultades de inserción, actualización y eliminación de roles quedan blindadas exclusivamente para el rol `'admin'`.

Políticas aplicadas a Books (Catálogo de Libros)

- **Lectura (SELECT):** Acceso general autorizado para todos los usuarios autenticados, permitiendo la exploración libre del catálogo.
- **Escritura (INSERT, UPDATE, DELETE):** Operaciones estrictamente denegadas para usuarios regulares. Permiso exclusivo para cuentas con rol verificado de administrador (`'admin'`).

Políticas aplicadas a Loans y Reservations (Préstamos y Reservas)

- **Usuarios con Rol 'user':** Autorizados únicamente a visualizar sus propios registros históricos de préstamos y reservas (`auth.uid() = user_id`), y con permisos específicos para registrar de forma autónoma nuevas inserciones de reserva bajo su propia identidad.
- **Usuarios con Rol 'admin':** Gozan de una política de acceso libre (`ALL`) sobre estas entidades, permitiéndoles auditar, modificar estados (por ejemplo, transicionar de 'active' a 'overdue' o de 'pending' a 'ready') y gestionar las transacciones de cualquier usuario del sistema.

7. Triggers y Automatizaciones

Para asegurar la máxima consistencia e integridad referencial independiente de las acciones del frontend, se han programado procedimientos operativos automáticos (Disparadores) directos en el motor de base de datos:

A. Automatización de Auditoría Temporal

Con el fin de registrar con precisión cronológica cada alteración física de los datos de negocio, se dispone del disparador generalizado de actualización. Este intercepta las sentencias de tipo `BEFORE UPDATE` en las tablas `profiles`, `books`, `loans`, y `reservations`, sustituyendo el valor del campo `updated_at` por la estampa de tiempo en tiempo real del servidor mediante la función procedural:

```
CREATE OR REPLACE FUNCTION public.update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = now();
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;
```

B. Orquestación Automática en el Registro de Usuarios (Onboarding)

Para suprimir la necesidad de peticiones API concurrentes durante el registro de nuevos usuarios, la base de datos se encuentra sincronizada mediante el disparador `on_auth_user_created`, configurado para reaccionar inmediatamente `AFTER INSERT ON auth.users`. Este ejecuta en un entorno asíncrono y seguro (*Security Definer*) las siguientes acciones secuenciales:

1. Extrae los metadatos capturados en el proceso de inscripción (tales como el nombre provisto en `raw_user_meta_data->>'name'` y el correo electrónico `email`) y los inserta de manera estructurada en la tabla de negocio `public.profiles`.
2. Inserta de manera predeterminada y blindada un registro en la tabla de control de accesos `public.user_roles` asignando el nivel de privilegios inicial correspondiente al rol de usuario estándar (`'user'`).

Este flujo automatizado garantiza un entorno íntegro, homogéneo y listo para operar desde el primer instante en que una cuenta es dada de alta en el sistema.